SMS-FLOW-001 · INTERNAL

Project Flow

Society Management System

This document describes how the running application works: who signs in, which screens they see, and how gate, billing, helpdesk, and society setup move from one step to the next.

Document date	8 October 2026
Audience	Product owners, developers, and testers
Application	React web app and Laravel API
Scope	`frontend/` and `backend_staging/`
Login	http://127.0.0.1:5173/login
API	http://127.0.0.1:8002/api/v1

Contents

1. Project overview
2. Sign-in and navigation
3. Gate flow
4. Billing flow
5. Helpdesk flow
6. Society and units flow
7. End-to-end demonstration

1. Project overview

The Society Management System is a resident welfare association application. One platform can hold many societies. Inside a society, staff manage flats, maintenance bills, complaints, and the gate.

People

Role	Who they are	What they do
Super Admin	Platform operator	Creates societies and switches the working society. Can use every admin and gate screen.
Society Admin	RWA office	Manages flats, invoices, payments, complaints, passes, and vehicles for one society.
Resident	Flat owner or tenant	Sees their own bills, ledger, complaints, visitor passes, and vehicles.
Guard	Gate staff	Checks visitors in and out, and redeems pre-approved passes.

Seeded demo users belong to **Green Valley Residency** (Pune). A second society, **Lakeview Apartments** (Mumbai), exists so the Super Admin can switch context.

How a request moves

> **1. Browser.** The person uses the React app at `http://127.0.0.1:5173`.

> **2. API.** The app calls `/api/v1` on `http://127.0.0.1:8002` with a bearer token.

> **3. Database.** Laravel stores the record in SQLite (current local setup) or MySQL.

Society is the boundary

Almost every record carries a society id. The API uses the signed-in user's current society and returns only that society's data. Society Admin, Guard, and Resident stay in the society on their account. Super Admin chooses the working society from the header. Residents are narrowed again: bills, ledger, and vehicles follow their flats; complaints and passes follow records they created.

Screen map

Screen	Path	Super Admin	Society Admin	Resident	Guard
Login	`/login`	Yes	Yes	Yes	Yes
Dashboard	`/`	Yes	Yes	Yes	Yes
Gate console	`/guard`	Yes	Yes		Yes
Visitor passes	`/visitor-passes`	Yes	Yes	Yes	
Vehicles	`/vehicles`	Yes	Yes	Yes	
Invoices	`/invoices`	Yes	Yes	Own flats	
Ledger	`/ledger`	Yes	Yes	Own flats	
Helpdesk	`/complaints`	Yes	Yes	Own tickets	
Units	`/units`	Yes	Yes		
Societies	`/societies`	Yes			

A person who opens a path their role cannot use is sent back to the dashboard. The API enforces the same role list on every route.

2. Sign-in and navigation

> **1.** The person opens `/login`.

> **2.** The app sends the email in lowercase and the password with surrounding spaces removed to `POST /api/v1/login`.

> **3.** A matching account receives a Sanctum token named `api-token`, plus name, role, and current society.

> **4.** The token is kept in the browser and sent as `Authorization: Bearer {token}`.

> **5.** The dashboard opens. On a later visit, a saved token loads the profile with `GET /api/v1/user`. A rejected token clears the session and returns to login.

> **6.** Logout calls `POST /api/v1/logout`, deletes that token, and clears the browser session.

A wrong email or password returns “Invalid credentials.” and stays on the login page.

Society context

> **Super Admin.** The header switcher saves the chosen society with `PUT /api/v1/user/current-society`. Later lists follow that society. Until a society is chosen, society-scoped actions cannot run.

> **Society Admin, Guard, and Resident.** They use the society already stored on the account. They cannot switch society.

Menu after login

Role	Menu
Guard	Dashboard and Gate console.
Resident	Invoices, Ledger, Helpdesk, Passes, and Vehicles, limited to their own flats and their own tickets and passes.
Society Admin	Resident screens, plus Units and the Gate console. They create invoices, record payments, update complaints, and attach people to flats.
Super Admin	Every Society Admin item, plus Societies and the header society switcher.

Demo accounts

> These accounts are for the local demonstration database. The password for every account is `password`.

Role	Email	Phone
Super Admin	`super@demo.local`	9000000000
Society Admin	`admin@demo.local`	9000000002
Guard	`guard@demo.local`	9000000001
Resident (owner of A-101)	`resident@demo.local`	9000000003

3. Gate flow

Guards, Society Admins, and Super Admins use the gate console. Residents create passes and register vehicles. They do not open the gate console.

Walk-in visitor

1. The console loads people whose status is entered (`GET /api/v1/visitors?inside_only=1`).

2. The guard enters name, phone, and purpose, and chooses guest or delivery. A delivery can include a vendor name. A flat and vehicle number are optional.

3. `POST /api/v1/visitors/check-in` stores the visitor on the current society, records who checked them in, and sets status to entered.

4. If a flat is chosen, it must belong to the same society. A visit can also have no flat.

5. The name appears under Currently inside.

6. Check out sets status to exited, stores the time, and records who checked them out. A second check-out is rejected.

Pre-approved visitor pass

1. A resident or admin opens Passes and enters guest name, phone, flat, valid from, valid until, and maximum entries.

2. A resident can create a pass only for a flat linked to their account.

3. The API creates an 8-character pass code with status active and entries used at 0.

4. Residents see only passes they created. Admins see every pass in the society. The creator or an admin can revoke a pass.

5. At the gate, staff redeem the code. It must belong to the current society, be inside the valid window, still be active, and have remaining entries.

6. A successful redeem checks the guest in with purpose “Pre-approved guest visit” and adds one to entries used.

7. When used entries reach the maximum, the pass becomes expired. An invalid, expired, revoked, or fully used code checks nobody in.

8. The guest leaves through the normal check-out button.

Vehicles

A resident or admin opens Vehicles and registers a plate on a flat. The type is car, bike, or other, with an optional label. A resident can register a vehicle only on their own flats. Plate numbers are unique inside a society. Deleting a vehicle removes it from that flat’s list. The register does not open the gate by itself. A walk-in can still type a vehicle number on the check-in form.

4. Billing flow

Society Admins and Super Admins create charges and record payments. Residents read invoices, payments, and ledger lines for flats they belong to.

Charge a flat

> **1.** An admin opens Invoices and submits flat, invoice number, amount, and due date.

> **2.** The flat must belong to the current society, and the invoice number must be unique in that society.

> **3.** The invoice is saved as unpaid.

> **4.** Accounting posts a ledger debit for the same amount, described as “Maintenance invoice {number}”, with reference type invoice.

> **5.** The resident of that flat sees the invoice on their Invoices page.

Record a payment

> **1.** An admin records a payment against one unpaid invoice.

> **2.** Methods are cash, bank transfer, UPI, and Razorpay. Notes and a gateway reference are optional.

> **3.** The amount must match the invoice total. Partial payments are not accepted. A paid invoice cannot be paid again.

> **4.** The payment is saved as completed, the invoice is marked paid, and a ledger credit is posted with reference type payment.

Ledger

Event	Debit	Credit	Reference
Invoice created	Invoice amount	0	invoice
Payment completed	0	Payment amount	payment

Admins see the society’s ledger. A resident sees lines for their flats only.

Razorpay

Online collection is prepared and is not required for the demo. Keys are blank. A payment can be stored as method razorpay with status pending. Razorpay later calls `POST /api/v1/webhooks/razorpay`. When a webhook secret is set, the signature header must match. The event `payment.captured` completes a pending payment with that gateway id, marks the invoice paid, and writes the ledger credit once. Cash, bank transfer, and UPI payments recorded by an admin complete immediately.

5. Helpdesk flow

Residents and admins can open a ticket. Only Society Admins and Super Admins can assign it or change its status.

Status	Meaning
open	Newly filed. This is the starting status.
in_progress	Someone is working on it.
resolved	Closed. The resolved time is set. Moving it off resolved clears that time.

1. The reporter opens Helpdesk and submits a title, description, and category.

2. The ticket is stored on the current society, under the reporter, with status open.

3. A resident sees only tickets they filed. An admin in the same society sees every ticket. The list can be filtered by status.

4. An admin may assign the ticket. The assignee must be a Society Admin or Super Admin of this society.

5. Setting status to resolved stamps the resolved time. Any other status clears it.

6. A resident cannot change status or assignment. A ticket from another society is hidden.

6. Society and units flow

A society owns flats. Flats own members. Residents then see billing, passes, and vehicles for those flats.

Societies

Only the Super Admin opens Societies.

1. Create a society with name, address, and registration number. The registration number is unique when present.
2. The same screen can update or delete a society.
3. Choosing a society in the header saves it on the Super Admin account.
4. Gate, invoices, complaints, units, passes, and vehicles then use that society.

Flats

Society Admin and Super Admin open Units. A flat has block, flat number, floor, and status occupied or vacant. Block plus flat number must be unique inside the society. Gate check-in and invoice creation both refuse a flat from another society. Guards can list flats for check-in and cannot create or edit them.

Society	Block	Flat	Floor	Status	Member
Green Valley Residency	A	101	1	Occupied	Flat 101 Resident, owner
Green Valley Residency	A	102	1	Vacant	None

Lakeview Apartments is seeded with no flats, for the society switcher.

Members

1. An admin opens a flat and attaches a user as owner or tenant.
2. They can detach that member later.
3. After attachment, that resident's invoices, ledger, and vehicles include the flat.
4. Visitor passes the resident creates must use one of their flats.

7. End-to-end demonstration

Use a fresh browser session, or log out, when changing role. Each demo email is a separate account. Password for each is `password`.

Step	Who	Action	Result
1	Super Admin	Sign in, open Societies, switch to Green Valley Residency.	Working society is set.
2	Society Admin	Open Units and confirm A-101 and A-102.	Flats exist.
3	Society Admin	Create an invoice for A-101.	Unpaid invoice and a ledger debit.
4	Resident	Sign in and open Invoices and Ledger.	They see that charge.
5	Society Admin	Record a full payment.	Invoice paid and a ledger credit.
6	Resident	Open Helpdesk and file a complaint.	Ticket is open.
7	Society Admin	Set the complaint in progress, then resolved.	Ticket closes with a resolved time.
8	Resident	Create a visitor pass for A-101.	Pass code is active.
9	Guard	Redeem the code on the Gate console.	Guest is inside.
10	Guard	Check the guest out.	Guest has exited.

Start the API with `php artisan serve --host=127.0.0.1 --port=8002` from `backend_staging`, and the web app with `npm run dev` from `frontend`. Both can also be started with `scripts/run-dev.ps1`.